
第 10 章

Agent workflow与验证方法

本章配套：正文小节 / 上机 lab / 练习 quiz / 常见误区与技巧

10. Agent 工作流与验证方法

本页是第二部分第 10 章的章首页。上一章回答“它是什么、怎么装”，本章回答“怎么干才靠谱”。

本章概览

Agent 编程的成败，很少取决于模型强弱，而取决于**方法与验证**：有没有把项目规范写进 AGENTS.md、复杂任务有没有先出计划、有没有把“什么叫做对”写成可执行的判据、关键结论有没有第二双眼睛。

本章把这套方法固化成可重复的操作流程，并在最后一节给出科研规范与学术诚信要求（AI 使用声明）。

学习目标

- 1. 会写最小可用的 AGENTS.md，把项目约定变成常驻指令；
- 2. 会用计划模式推进复杂任务，并用“小步执行 + 逐步验证”控制返工；
- 3. 会用**任务卡六件套**（目标、约束、验收标准、证据、复核、提交）描述任务；
- 4. 会为科学计算任务写出**可执行的验收标准**，并用断言把结论钉死；
- 5. 知道什么时候该派子代理、什么时候纯属浪费；
- 6. 说出 12 条常见误区，并按规范提交 AI 使用声明。

本章结构

小节	内容	状态
01 项目级指令与计划模式	AGENTS.md、计划先行、会话与上下文、权限约定固化	已交付
02 任务卡与验收标准库	任务卡六件套、验收标准正反例、第 1–8 章判据库、提示词清单	已交付
03 子代理、技能与工具	工具与技能、子代理判据、协作规矩与成本	已交付
04 常见误区与科研规范	12 个误区、AI 使用声明模板、可复现三件套、学术诚信	已交付
02 任务卡与验收标准库	六件套、“验收标准写法的正反例”、面向第 1–8 章的判据库	编写中
03 子代理、技能与工具	角色分工、成本、技能沉淀	编写中
04 常见误区与科研规范	12 条误区、AI 使用声明模板、可复现三件套、学术诚信红线	编写中

资源导航

资源	位置	说明
本章正文	01 · 02 · 03 · 04	四节构成完整方法链
参考资料	references.md	官方文档与延伸阅读
教学说明	teaching.md	教师用：课时、案例拆解、评分要点
配套案例	../11-agent-cases/README.md	本章方法的落地演练

与前后章的关系

- 上一章（第 9 章）解决“是什么、怎么装、怎么安全用”；
- 本章解决“方法”；
- 下一章（第 11 章）用四个递进案例把方法跑一遍；
- 第 12 章解决“长期怎么维护”（版本、成本、协作、项目）。

01 项目级指令与计划模式

本节解决“怎么让 Agent 一开始就按你的规矩干活”：把约定写进 AGENTS.md、复杂任务先出计划、长任务管好上下文。权限与安全的完整实践见第 9 章 04 节；本节只讲“如何把权限约定固化下来”。

本节目标

1. 会写一份最小可用的 AGENTS.md，让 Agent 不用每次重复问规范；
2. 会用“计划先行 + 小步执行”推进复杂任务；
3. 会管理会话与上下文：一次一件事、阶段收口、换任务开新会话；
4. 会把权限与数据约定写进项目文件，而不是靠每次口头提醒。

先修

- 第 9 章 01-04 节（概念、安装、界面与斜杠命令、权限与数据红线）。

1.1 项目级指令：AGENTS.md

AGENTS.md 是放在项目里的“常驻说明”，Agent 每次工作都会先读它。它解决的是**重复解释**问题：与其每次都读 CSV 要指定 UTF-8、图存到 output/、随机种子固定 42，不如写进文件。

最小可用模板（复制到项目根目录即可）：

项目说明

本项目是《Python 科学计算》课程的数据分析练习。

环境

- Python 3.12, 依赖见 environment.yml (numpy / pandas / scipy / statsmodels / scikit-learn / matplotlib)
- 运行命令统一用 python < 脚本名 >

约定

- 读 CSV 一律显式指定编码 utf-8-sig, 并先打印列名
- 随机过程必须固定种子: np.random.default_rng(42)
- 输出文件放 output/, 不覆盖 data/ 里的原始数据
- 中文图必须设置字体: Microsoft YaHei 或 SimHei

安全 (对应第 9 章 04 节)

- 只允许改工作区内的文件；删除、批量替换、装依赖前必须先问我
- 不读取 .credentials.yaml / .env 等凭据文件，不把它们的内容写进任何输出

质量要求

- 每个脚本末尾要有 assert 自检；改动后必须运行一次并贴出输出
- 不确定的地方先问我，不要擅自扩大改动范围

三条经验：

1. **只写“会被违反”的约定**：写得越长越没人看，Agent 也一样；
2. **写清验收方式**：能让它自查的约定，比形容词有用；
3. **把 AGENTS.md 提交到 git**：它是项目的一部分，不是个人偏好。

1.2 计划先行：复杂任务先要计划

任务超过“改一个文件”的量级时，先让它进入**计划模式**（界面里的 /plan，见第 9 章 03 节），产出：要动哪些文件、分几步、每步怎么验证、有什么风险。你确认后再动手。

任务规模	建议做法
一行改动、明确的小修	直接做，人审 diff
单文件重构、补测试	先说明要做什么，再执行
多文件、跨目录、动数据	必须计划模式 + 你确认
涉及删除、覆盖、联网安装	计划 + 逐条确认

计划模式的价值不是“礼貌”，而是把**返工成本**从“改完发现方向错”降到“看计划发现方向错”。

1.3 会话与上下文管理

- **一次会话只做一件事**：上下文里塞太多目标，它就容易抓错重点；
- 长任务定期收口，让它输出“已确认的结论 / 未解决的问题 / 下一步”——这三行就是压缩后的上下文；
- 换任务就开新会话，把上一轮结论作为开场信息带过去；
- 会话过长时用 /compact，但**重要结论要落到文件里**（如 output/review.md）；
- 会话记录是**证据链**：提交作业时用 /export 导出或截图留档。

1.4 把权限与数据约定固化下来

审批是“事后同意”，AGENTS.md 是“事前约定”。两者配合：

场景	写在 AGENTS.md 里的约定
原始数据	只读，不修改、不重命名 data/ 下的文件
输出产物	一律写到 output/，文件名固定（便于验收）
危险动作	删除、批量替换、安装依赖、联网前必须先说明并获得同意
凭据	不读取、不打印、不写入任何凭据文件

本节小结

1. AGENTS.md 把“每次都要说的规范”变成项目文件，并纳入版本管理；
2. 复杂任务先计划、后执行、小步验证；
3. 会话一次一件事，阶段收口，重要结论落盘；
4. 权限与数据约定要写进文件，而不是靠记忆。

思考题

1. 为什么“只写会被违反的约定”比“写得越全越好”更有效？
2. 计划模式在什么情况下反而是负担？请举一个具体例子。
3. 如果一个约定已经写进 AGENTS.md，但 Agent 仍然违反了，你会怎么排查？

练习与延伸阅读

- 练习：第 11 章 `exercises/` 第 6 题（为你的项目写 `AGENTS.md`）；
- 下一节：[02-任务卡与验收标准库.md](#)。

02 任务卡与验收标准库

本节是全部分的核心方法：把任何任务写成**可验收**的任务卡，并为科学计算任务准备一套现成的判据库。

本节目标

- 1. 会用**任务卡六件套**描述任务；
- 2. 会把形容词式要求改写成**可执行断言**；
- 3. 能从“验收标准库”里为第 1–8 章的典型任务挑选判据；
- 4. 会用提示词七条清单把任务说清楚。

先修

- 第 9 章（Agent 循环与失败模式）、本章 01 节（AGENTS.md 与计划模式）。

2.1 任务卡六件套

要素	写什么	例
目标	一句话说清要达成什么	算出每个学生的平均分
约束	不许动什么、用什么环境	只新增 scores_fixed.py, 不动 data/
验收标准	可执行 的判据	长度 8、形状 (8,)、与手算一致
证据	要它给出什么证明	运行输出 + 断言结果
复核	人要检查哪几点	归一化是否逐列、排名是否带姓名
提交	交付形态	文件 + 说明 + 会话记录

2.2 验收标准的写法：从形容词到断言

差的写法	好的写法
结果要正确	平均分长度必须是 8，且与手算 [83.33, 65.0, ...] 一致
图画得好看点	图保存到 output/trend.png；x 轴为小时、y 轴为温度（摄氏度）；标题含“传感器温度”
检查有没有问题	用 assert 检查：行数守恒、缺失值策略已记录、归一化逐列
模型效果要好	5 折交叉验证最低分 ≥ 0.90 ，且预处理在 Pipeline 内

为什么必须可执行：只有可执行的判据才能被反复验证。同一段代码，两种 axis，只有一种能通过断言：

```
import numpy as np
np.set_printoptions(precision=2, suppress=True)
scores = np.array([[82, 90, 78], [60, 65, 70], [95, 88, 92], [70, 72, 68]], dtype=float)

def check_student_mean(m):
    m = np.asarray(m)
    assert m.shape == (4,), " 形状必须是 (4, ), 每个学生一个平均分"
    assert np.allclose(m, scores.mean(axis=1)), " 数值必须与手算一致"
    return True
```

```
for axis in (0, 1):
    try:
        check_student_mean(scores.mean(axis=axis))
        print("axis=%d -> 断言通过" % axis)
    except AssertionError as e:
        print("axis=%d -> 断言不通过: %s" % (axis, e))

axis=0 -> 断言不通过: 形状必须是 (4,)，每个学生一个平均分
axis=1 -> 断言通过
```

2.3 验收标准库（面向第 1–8 章）

写任务卡时，直接从下表挑判据，改成你的具体数值即可：

章节 / 任务类型	可执行判据（举例）
第 1 章 NumPy	形状与广播结果 (x.shape); 与手算/解析解对照 (np.allclose); 视图与拷贝 (np.shares_memory); 残差与条件数 (np.linalg.norm(A@x-b)、np.linalg.cond(A))
第 2 章 SymPy	符号结果数值代入验证 (expr.subs); 两式之差化简为 0 (simplify(a-b) == 0); 与数值方法交叉验证
第 3 章 SciPy	积分与解析解对照; 优化收敛信息 (res.success、res.fun 对比理论最优); 插值在节点处误差为 0; 检验 p 值与手算一致
第 4 章 pandas	行数守恒 (清洗前后差值 = 删除行数); 缺失策略留痕; 分组聚合与手算对照; 表格一致性 (assert_frame_equal)
第 5 章 Matplotlib	图保存路径与 dpi; 坐标轴标签与单位; 中文无乱码; 随机种子固定; 数据点数量与源数据一致
第 6 章 NetworkX	节点/边数与源数据一致; 连通性 (nx.is_connected); 最短路长度与手算对照; 社区划分的模块度
第 7 章 Statsmodels	系数符号与量级合理; p 值与置信区间; R² 与残差诊断 (Durbin-Watson); 重跑一致 (固定种子)
第 8 章 sklearn	交叉验证分数区间; Pipeline 结构检查 (防泄漏); 与基线对照 (DummyClassifier); 指标口径与混淆矩阵一致
通用工程判据	脚本可重复运行; 输出文件存在且非空; 断言全部通过; git diff 只涉及预期文件

用法：把“第 8 章那类任务”的判据**数字化**——例如“5 折最低分 ≥ 0.90 ” “行数守恒：11 $\rightarrow$ 10，删 1 行” “max(axis=0) 全为 1”。

2.4 提示词七条清单

- 1. **说清目标与判据**，不要只说“帮我看看”；
- 2. **给最小复现**：能跑的 20 行，胜过 2000 行项目；
- 3. **限定范围**：明确“只改 X、不动 Y”；
- 4. **要求自证**：让它跑一次并贴输出，而不是“我觉得对”；
- 5. **要求打印中间量**：形状、量纲、前几行、统计量；
- 6. **一次只做一件事**：多个目标拆成多轮；
- 7. **不确定就问**：明确告诉它“信息不足时先提问，不要猜”。

2.5 一个完整示例

【目标】修复 `data/scores_buggy.py` 中“每个学生平均分”和“归一化”两处错误，产出 `data/scores_fixed.py`。

【约束】只新增 `scores_fixed.py`，不修改 `data/scores.csv` 与 `scores_buggy.py`；使用现有 `scicomp` 环境。

【验收标准】

- 1) `student_mean` 的形状是 `(8,)`；
- 2) 归一化逐列进行：`norm.min(axis=0)` 全为 0、`norm.max(axis=0)` 全为 1；
- 3) 加权总分第一名是王五（91.4 分），且打印时带姓名。

【证据】贴出 `python data/scores_fixed.py` 的完整输出，并给出三条 `assert` 的通过结果。

【复核】我会检查：是否动了原始数据、归一化是否逐列、排名是否真的排序。

【提交】`scores_fixed.py` + 运行输出 + 本次会话记录。

本节小结

1. 任务卡六件套：目标、约束、验收标准、证据、复核、提交；
2. 验收标准必须可执行，否则等于没有；
3. 判据库把第 1–8 章的知识变成了“AI 交付物的检查清单”；
4. 提示词七条清单解决“说不清”的问题。

思考题

1. 把“报告要有说服力”改写成两条可执行判据；
2. 第 7 章的回归任务里，哪三条判据可以防住“看起来显著其实不可靠”？
3. 为什么“要求它贴出运行输出”比“要求它保证正确”更有效？

练习与延伸阅读

- 练习：第 11 章 `exercises/` 第 7–8 题（任务卡与判据改写）；
- 下一节：[03-子代理、技能与工具.md](#)。

03 子代理、技能与工具

本节回答“能力怎么扩展、任务怎么分工”：工具与技能的区别、什么时候派子代理、怎么让它们不互相打架。

本节目标

- 1. 说清工具 (tools) 与技能 (skills) 的区别，知道什么时候值得沉淀技能；
- 2. 会用三条判据决定是否派子代理；
- 3. 会规定子代理的输入与汇报格式，避免“合并时才发现对不上”；
- 4. 知道子代理的成本来自哪里。

先修

- 第 9 章 01 节 (harness 五件套)、本章 01-02 节 (AGENTS.md 与任务卡)。

3.1 技能与工具

	工具 (tools)	技能 (skills)
粒度	原子动作：读文件、写文件、跑命令、搜索	一组动作 + 领域知识的打包
例子	read、write、bash	“按模板生成实验报告”“批量处理 Excel”
何时用	每次任务都会用到	某套流程你会重复做三遍以上、且步骤固定
谁来定义	harness 提供	你（或团队）沉淀，写进项目或插件

沉淀技能的最小形式其实很简单：**一份模板文件 + AGENTS.md 里的一段说明**。等它稳定了，再考虑做成正式技能。

3.2 子代理：什么时候值得拆

场景	是否拆	原因
先摸清陌生代码库，再决定怎么改	拆（探索代理）	大量阅读会污染主上下文
三个互不依赖的模块同时改	拆（并行）	省时间
关键结论需要第二双眼睛	拆（评审代理）	专职找错，避免自我确认
改一个函数、加一行日志	不拆	派发与合并的开销比任务本身还大

记住：子代理不是“更强的模型”，而是**上下文隔离 + 角色分工**；每个子代理都会单独消耗 token。

3.3 派子代理的三条规矩

- 1. **给明确的输入边界**：探索代理只给目录与问题；评审代理只给 src/ 与 output/，**不给你的解释**（否则它会顺着你的思路走）；

子代理分工：主代理负责合并与验收

适合探索、并行、评审；小任务不必拆

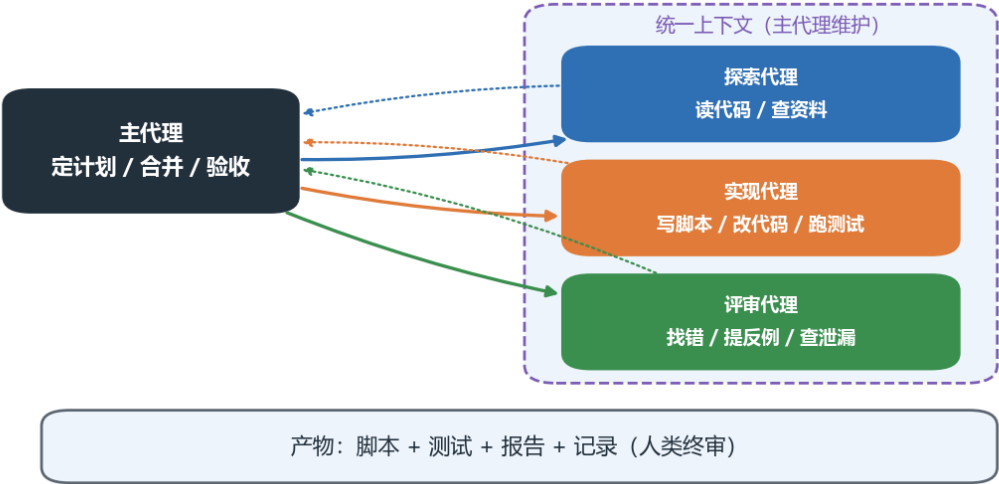


Figure 1: 子代理分工

2. **规定汇报格式**：例如评审代理必须输出表格“问题 / 证据（文件: 行） / 建议修法 / 严重程度”，便于你逐条回应；
3. **主代理负责合并与验收**：子代理不直接改最终交付物，避免并发覆盖与责任不清。

3.4 workflow模板：探索 → 实现 → 评审

【角色分工】

- 1) 探索代理：只读，输出“数据概况 + 3 条建模建议”，附证据（文件名/行号）。
- 2) 实现代理：按任务卡改代码，必须运行并贴输出。
- 3) 评审代理：只看 src/ 与 output/，输出 ≥ 3 条质疑（表格），不得修改文件。

【主代理】合并产物、逐条回应质疑、给出最终结论与局限。

【人】确认计划、检查 diff、签署结论。

3.5 成本提示

- 一个子代理 = 一份独立的上下文与模型调用；
- 并行三个子代理 $\approx$ 三倍 token；
- 小任务派子代理，往往比直接做更慢也更贵。

本节小结

1. 工具是原子动作，技能是打包的工作流；重复三次以上就该沉淀；
2. 只在“污染上下文、可并行、需评审”三类场景派子代理；
3. 输入边界与汇报格式要事先约定，合并与验收归主代理；
4. 子代理是真金白银的 token，先算账再拆。

思考题

1. 为什么评审代理不该拿到你的解释？
2. 什么情况下“两个子代理并行”反而会拖慢进度？
3. 你打算把哪套重复流程沉淀成技能？写出它的输入、输出与验收标准。

练习与延伸阅读

- 练习：第 11 章 exercises/ 第 8 题（为案例四设计子代理分工）；
- 下一节：[04-常见误区与科研规范.md](#)。

04 常见误区与科研规范

本节是第二部分的方法收口：先列出最常见的 12 个误区，再给出可执行的科研规范（AI 使用声明、可复现三件套、学术诚信红线）。

本节目标

- 1. 识别 12 个常见误区，并说出对应的纠正做法；
- 2. 会写一份合格的 **AI 使用声明**；
- 3. 掌握可复现三件套（版本锁定、随机种子、环境记录）；
- 4. 明确学术诚信的允许与禁止边界，以及“什么时候不该用 Agent”。

4.1 十二个常见误区

#	误区	典型表现	纠正做法
1	把 Agent 当搜索引擎	只问“怎么做”，不干活	给任务卡与判据，让它执行并自证
2	不给验收标准	“帮我优化一下”	任务卡六件套，判据可执行
3	一次要求改太多	一句话让它重构整个项目	拆成多轮，每轮一个目标
4	不看 diff 就接受	直接 git commit	每步看 git diff，确认范围
5	用“跑通”代替“正确”	脚本能跑就算完	对照判据库，跑断言
6	盲信图表	图好看就下结论	检查坐标轴、单位、样本量、异常点
7	忘记固定种子	别人复现不了	np.random.default_rng(42)，写进 AGENTS.md
8	忘记记录版本	三个月后无法复现	记录 Python/库/dsh 版本（可复现三件套）
9	把 Key 写进 notebook	提交时泄漏	用凭据文件；提交前 git status 检查
10	让它直接改原始数据	data/ 被就地覆盖	原始数据只读，输出写到 output/
11	评审缺位	自己写、自己评、自己签	引入评审代理或同学互评
12	把隐私数据贴进对话	成绩单、手机号、医疗记录	脱敏或本地处理；红线不可越

4.2 AI 使用声明（模板）

提交作业与报告时附上，逐条填写（不能只写“使用了 AI 辅助”）：

- ## AI 使用声明
1. 使用的工具与版本：DeepSeek Harness < 版本 >，模型 <deepseek-flash / deepseek-v4-pro>，日期 <YYYY-MM-DD>
2. 使用环节：☐ 环境搭建 ☐ 代码编写 ☐ 调试排错 ☐ 数据分析 ☐ 图表 ☐ 文字润色 ☐ 其他：_____
3. 关键提示词：附 2-3 条最能代表工作方式的提示词（或随附 /export 的会话记录）
4. 人工修改：说明你对 AI 产出做了哪些实质性修改（至少 3 条）
5. 验证方式：列出你运行的断言/复核（如行数守恒、交叉验证、与手算对照）
6. 未使用 AI 的部分：问题定义、结论、报告正文（按课程要求勾选）
7. 我确认：最终提交内容由我负责，AI 产出已逐条核校。

4.3 可复现三件套

要素	做法	检查方式
版本锁定	记录 Python 与小包版本；dsh 记录基线版本（见 docs/build/dsh_version.yaml）	报告里列出 python -V、关键库版本、dsh --version
随机种子	所有随机过程固定种子并写进 AGENTS.md	两次重跑结果一致（断言 np.allclose）
环境记录	依赖文件（environment.yml / requirements.txt）+ 运行命令	换机器按文件能跑通

4.4 学术诚信：允许与禁止

允许（鼓励）	禁止
用 Agent 解释报错、查文档、写样板代码	直接提交 AI 产出而不核验
用它生成测试、补边界条件，你审 diff	伪造数据、伪造运行结果
用它做初步分析，你复核统计结论	把 AI 生成的参考文献当作真实文献引用
在声明中如实记录使用过程	隐瞒使用（课程明确要求声明时）

判断标准：“如果老师问我这条结论是怎么来的，我能完整讲清楚并复现吗？”

4.5 什么时候不该用 Agent

- 一行代码、明确的小改动：自己写更快；
- 涉及未发表/涉密数据，且没有合规的本地方案；
- 你已经知道答案、只是想要“确认”：请用文档或同学互评；
- 时间紧到没空验收：没有验收的 AI 产出，风险高于收益。

本节小结

1. 12 个误区可以归纳为两条：不给判据、不做验收；
2. AI 使用声明要具体到版本、环节、提示词、修改与验证；
3. 可复现三件套：版本、种子、环境；
4. 学术诚信的底线是“能讲清楚、能复现、能负责”。

思考题

1. 从 12 个误区中挑 3 个你最可能犯的，写下你的预防措施；
2. 为什么“AI 生成的参考文献”是高危项？如何核验？
3. 如果课程不允许使用 AI，但你已经用了，应该怎么处理？

练习与延伸阅读

- 练习：exercises/assignment.md 第 11–12 题（项目复盘 + AI 使用声明）；
- 参考：本课程《期末大作业》说明与报告模板（教学资源/期末大作业/）。